LINKED LIST
PROBLEM 3

QUESTION:

DS	**Session**	Linked List	**Question Information**	Level 1 Challenge 33

Question description

Varman's Dream came true after he got an Appointment order from Google.Simon's family was very happy of his achievement.

The company mentioned Basic Salary, DA, HRA with some other benefits.

But not highlighted the Gross salary in the order.

varman's father wanted to know the Gross salary of his son.

varman try to his gross salary from HR department. they informed that you have to get pass grade in first month entry test. the entry test has 5 questions. one of the question was, Sorted insert in circular linked list.

Can you help varman?

Function Description

First case one is if linked list is empty then since new_node is only node in circular linked list, make a self loop.and change the head pointer to the new_node pointer.

Second case is new node insert in starting or before the head node.

A- Find out the last node using a loop .
```
While(present->!=*head_ref)
present=present->next;
```
B- Change the next of last node;
```
present->next=new-node;
```
C- Change next of new node to point to head.
```
new_node->next=*head_ref;
```

D- Change the head pointer to point to new node.
```
*head_ref=new_node;
```

Third case is when we insert the new node after the head in any position,then

A- Locate the node after which new node is to be inserted.
```
while(present->next!= *head_ref &&
present->next->data data)
{ present = present->next; }
```
B- Make next of new_node as next of the located pointer
```
new_node->next = present->next;
```
C- Change the next of the located pointer
```
present->next = new_node;
```

Constraints

0<n<100

Input Format:
The First line of the input represents the number of elements

Second line represents the elements of circular linked list

Output Format:
single line prints the results as per sample test cases

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
6
10 20 30 45 34 56
```

EXPECTED OUTPUT

```
10 20 30 34 45 56
```

Test Case 2

INPUT (STDIN)

```
6
1 2 11 12 56  90
```

EXPECTED OUTPUT

```
1 2 11 12 56 90
```

Mandatory Test Cases

Test Case 1	Test Case 2
KEYWORD	KEYWORD
struct Node *next;	void sortedInsert(struct Node** head_ref, struct Node* new_node)

Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
6	345	68

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

void sortedInsert(struct Node** head_ref, struct Node* new_node) {
    struct Node* current = *head_ref;
    if (current == NULL) {
        new_node->next = new_node;
        *head_ref = new_node;
    } else if (current->data >= new_node->data) {
        while (current->next != *head_ref)
            current = current->next;
        current->next = new_node;
        new_node->next = *head_ref;
        *head_ref = new_node;
    } else {
        while (current->next != *head_ref && current->next->data < new_node->data)
            current = current->next;
        new_node->next = current->next;
        current->next = new_node;
    }
}

int main() {
    int n, val;
    if (scanf("%d", &n) != 1) return 0;
    struct Node* head = NULL;
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
        new_node->data = val;
        new_node->next = NULL;
        sortedInsert(&head, new_node);
    }
    struct Node* temp = head;
    if (head != NULL) {
        do {
            printf("%d ", temp->data);
            temp = temp->next;
        } while (temp != head);
    }
    printf("\n");
    return 0;
}
```

OUTPUT:

```
6
10 20 30 45 34 56
10 20 30 34 45 56
```

```
6
1 2 11 12 56 90
1 2 11 12 56 90
```